



LSM Trees – arquitectura y casos reales

INTEGRANTES:

JUAN ANDRES RAMIREZ PATIÑO

JULIAN ESTEBAN HURTADO SERNA

JUAN PABLO OCAMPO SOTO



CONTENIDOS

01. Introducción

02. ¿Que es un lsm tree?

03. Importancia de los lsm tree

04. Diferencias con los B-Trees

06. arquitectura general

07. El motor en memoria

08. El motor en disco

10. El ciclo de vida

11. El mantenimiento

12. Pros y contras

13. Ejemplos y casos de uso

14. Conclusiones

¿QUE ES UN LSM TREE?

Preguntas de acercamiento

¿Es como un diccionario?

Estructura de los datos: clave → valor

¿Cuál es su objetivo principal?

Hacer **muchas** escrituras de datos de manera eficiente y rápida.

¿Donde opera?

Opera en la memoria RAM

¿Manejan un log de operaciones?

Sí, los árboles LSM utilizan un Write-Ahead Log (WAL) en disco antes de actualizar la memoria ram

¿Qué problema resuelve los LSM trees?

Resuelven principalmente el problema de las escrituras intensivas en disco, ofreciendo un rendimiento mucho mayor que estructuras tradicionales como los B-Trees, especialmente en sistemas con grandes volúmenes de datos y alta concurrencia.



¿QUE ES UN LSM TREE?

Definicion

Un LSM Tree, o Log-Structured Merge Tree, es una estructura de datos diseñada para optimizar escrituras masivas en sistemas de almacenamiento.

Analogía bibliotecario

Cuando llegan libros nuevos:

1. Los pone primero en una mesa rápida (RAM).
2. Cuando la mesa se llena:
 - los ordena,
 - los guarda en disco como un bloque.



IMPORTANCIA DE LOS LSM TREE

La importancia de los LSM Trees radica en que resolvieron un problema enorme de las bases de datos modernas:

¿Cómo guardar cantidades gigantescas de datos sin que las escrituras se vuelvan lentas?

¿Por qué son tan importantes?

Porque permiten:

Hizo posible muchas bases NoSQL modernas

Escribir datos MUY rápido

Ejemplo:

- mensajes de chat,
- likes,
- eventos,
- métricas,
- logs.

Aprovechan mejor los SSD

Los SSD funcionan mejor escribiendo de manera: secuencial

y peor con muchas modificaciones aleatorias pequeñas.

El LSM Tree justamente:

- acumula datos,
- escribe grandes bloques ordenados.

Son perfectos para Big Data

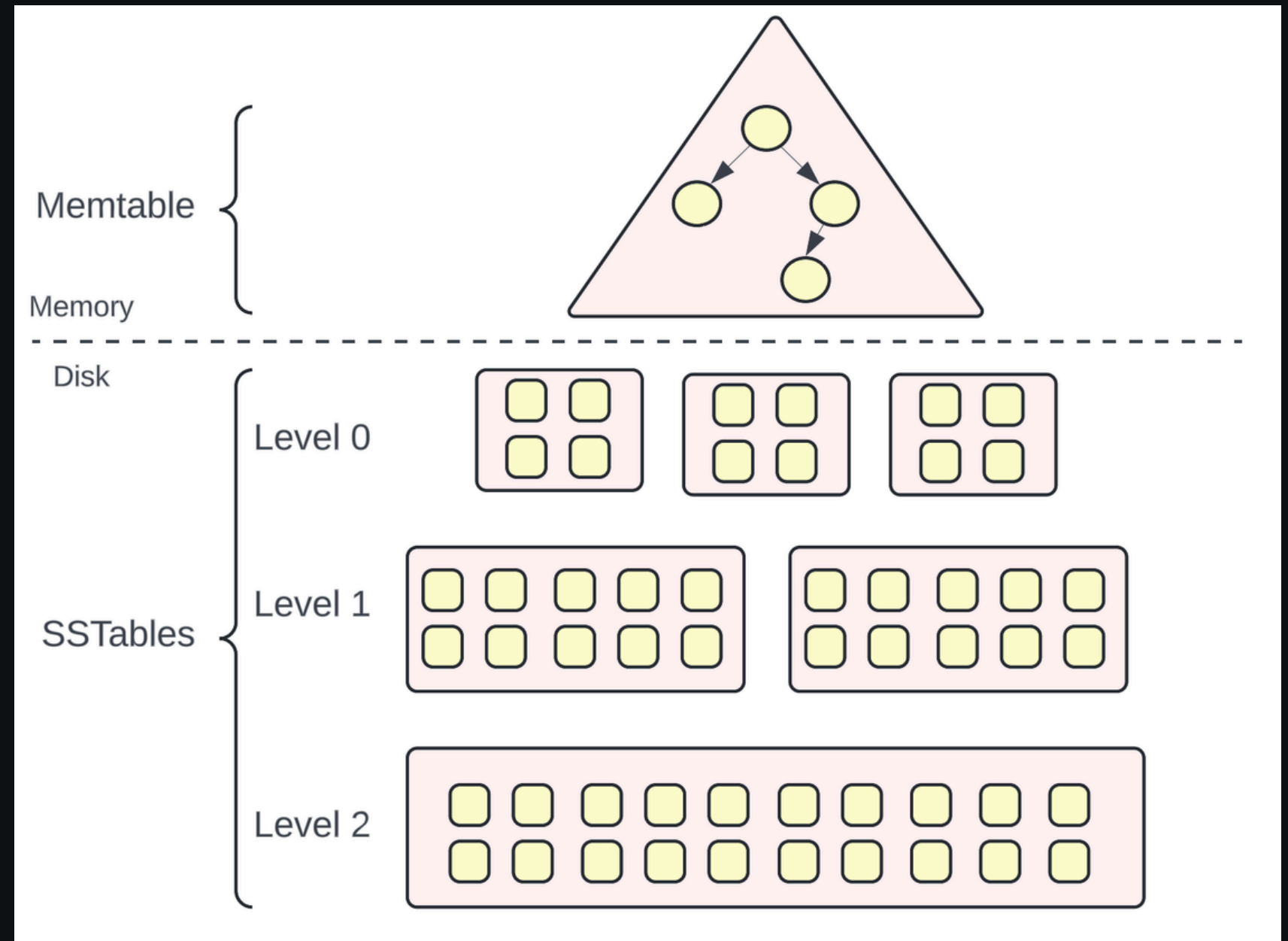


ARQUITECTURA GENERAL

Un LSM Tree es un árbol que combina:

- Log-Structured: cada operación de escritura se registra primero en un log secuencial en disco (Write-Ahead Log), garantizando durabilidad.
- Memtable: los datos se almacenan temporalmente en memoria en una estructura ordenada.
- SSTables: cuando la memoria se llena, los datos se vuelcan a disco en archivos ordenados, que luego se fusionan mediante compaction.

En conjunto, esto convierte escrituras aleatorias en escrituras secuenciales rápidas, manteniendo la capacidad de búsqueda y orden.



DIFERENCIAS CON LOS B-TREES

Tabla Comparativa **B-Tree** vs **LSM Tree**

Aspecto	B-Tree	LSM Tree
Idea principal	Mantiene todo ordenado en todo momento	Escribe rápido y ordena después
Velocidad de escritura	Más lenta	Muy rápida
Velocidad de lectura	Muy rápida	Puede ser más lenta
Cómo actualiza datos	Modifica el dato directamente	Guarda nuevas versiones y luego fusiona
Uso del disco	Escrituras aleatorias	Escrituras secuenciales
Estructura interna	Árbol balanceado clasico	Memoria + archivos SSTable
Fortaleza principal	Lecturas y consultas rápidas	Manejo masivo de escrituras
Ideal para	Bases de datos SQL tradicionales	Big Data y bases NoSQL

EL MOTOR EN MEMORIA (WRITE PATH)

MEMTABLE

- Estructura en memoria RAM (usualmente una Skip List o Red-Black Tree).
- Recibe las escrituras directamente y las mantiene ordenadas por clave en tiempo real.
- Latencia de escritura cercana a cero.

WAL (Write-Ahead Log)

- Registro secuencial en disco de cada operación.
- Tolerancia a fallos.



EL MOTOR EN DISCO (READ PATH)

SSTABLE (Sorted String Tables)

- Archivos en disco donde se guardan los datos de forma permanente.
- Son 100% inmutables.
- Los datos dentro de ellas están estrictamente ordenados por clave.

Filtros de Bloom (Bloom Filters)

- Estructura probabilística en RAM.
- Responde al instante si una clave no está en una SSTable, evitando accesos innecesarios al disco.

EL CICLO DE VIDA

Desde el Nacimiento hasta el Disco



ESCRITURA

El dato se registra en el WAL y se guarda ordenado en la MemTable (RAM).



FLUSH

Al llenarse la MemTable, se congela y se escribe secuencialmente en disco como una SSTable Nivel 0.



LIBERACION

Se vacía la MemTable, se borra el WAL viejo y el flujo continúa sin interrupciones



LECTURA

El sistema busca el dato en orden: MemTable → Filtros de Bloom → Índices de SSTables en disco.

EL MANTENIMIENTO

Limpieza y Consolidación del Disco

- Para eliminar datos duplicados antiguos y procesar las lápidas (tombstones).
- Para evitar tener demasiados archivos abiertos en disco.

¿Cómo funciona?

Un proceso en segundo plano (background) fusiona varias SSTables ordenadas en una sola (usando un algoritmo tipo Merge Sort), descartando lo obsoleto.

Estrategias Principales

- Size-Tiered
- Leveled



PROS Y CONTRAS

Pros:

- **Escritura Ultrarrápida:** Secuencial en lugar de aleatoria.
- **Excelente tasa de compresión**
- **Optimizado para SSDs**



Contras:

- **Amplificación de Lectura:** Hay que buscar en varios niveles.
- **Amplificación de Escritura:** La compactación reescribe datos varias veces.
- **Picos de Latencia**

EJEMPLOS Y CASOS DE USO



BASES DE DATOS QUE USAN LSM TREES

01

RocksDB (Meta):

Es una biblioteca de base de datos incrustada de alto rendimiento, optimizada para almacenamiento rápido y entornos de múltiples núcleos, diseñada para gestionar grandes volúmenes de datos clave-valor.

02

Apache Cassandra:

Es un sistema de gestión de bases de datos NoSQL distribuido, de código abierto y alta disponibilidad, diseñado para gestionar volúmenes masivos de datos en múltiples centros de datos sin puntos únicos de fallo.

03

ScyllaDB:

Es una base de datos NoSQL distribuida de código abierto, diseñada para ofrecer un rendimiento extremadamente alto y una latencia baja en aplicaciones de big data.

04

BigTable (Google):

Es un servicio de base de datos NoSQL de Google Cloud, totalmente administrado y de alto rendimiento, diseñado para cargas de trabajo analíticas y operativas a gran escala.

CASOS DE USO REALES

01

IoT y Sensores:

Millones de eventos por segundo volumen masivo de datos generados por dispositivos inteligentes. Debido a que las aplicaciones IoT generan secuencias continuas de datos.

02

Sistemas de Mensajería:

Chats masivos que generan un alto volumen de escrituras, como las aplicaciones de chat en tiempo real, registro de eventos y sistemas de mensajería distribuidos donde la persistencia rápida es crítica

03

Logs y Telemetría:

Guardar errores y métricas de servidores optimizadas para escritura,

04

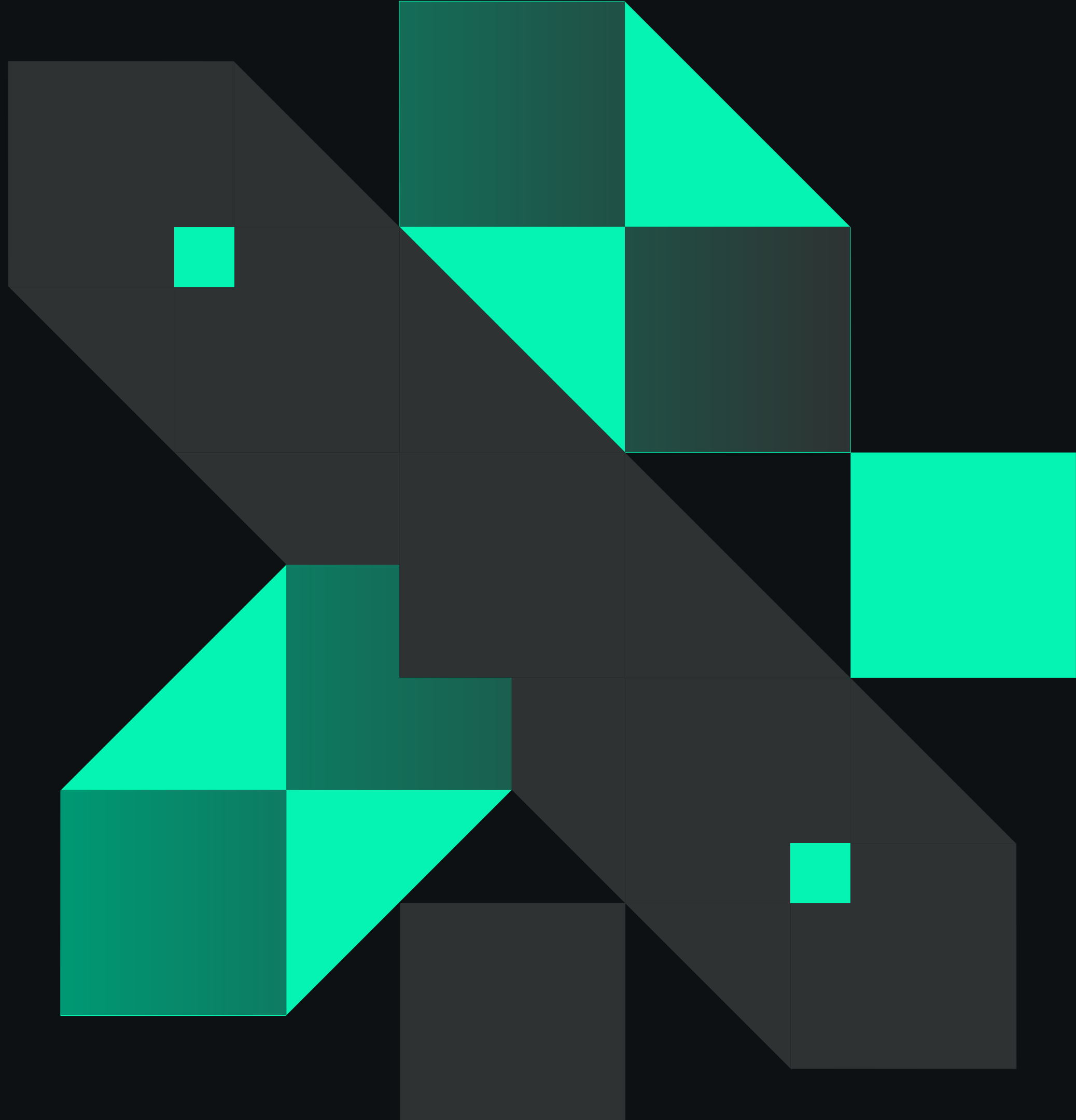
Blockchain:

Almacenamiento de transacciones inmutables
Permiten gestionar eficientemente millones de transacciones mediante componentes in-memory y archivos inmutables en disco.

CONCLUSIONES

- Los LSM Trees representan una de las soluciones más importantes para los sistemas modernos de almacenamiento de datos,
- Un aspecto clave es que su arquitectura basada en MemTables, WAL y SSTables permite mantener un equilibrio entre rendimiento, durabilidad y escalabilidad.
- Los LSM Trees son especialmente adecuados para aplicaciones que generan enormes cantidades de datos continuamente, como sistemas IoT, plataformas de mensajería, telemetría, logs y blockchain.

¡Gracias!





REFERENCIAS

- LSM: qué es y por qué es importante para las bases de datos. (2025, August 28). InnovaciónDigital360. <https://www.innovaciondigital360.com/big-data/lsm-para-empresas-beneficios-clave-y-aplicaciones/>
- Fiona. (2026). Estructuras de datos en disco: árboles B y LSM. Beefed.ai. <https://beefed.ai/es/on-disk-data-structures-btrees-lsms>